Home Control

Projekthandbuch für Schulabgabe, Bedienung und technischen Betrieb

1 Das Projekt auf einen Blick

Home Control ist eine Smart-Home-Web-App aus meiner Umschulung zum Softwareentwickler. Ich wollte die Geräte verschiedener Hersteller in einer Oberfläche sehen und ausgewählte Geräte schalten können. Als Beispiel verbindet eine Regel den Türkontakt mit dem Scharfschalten eines Blink-Systems.

> **Ausgangspunkt für die Vorführung:** Die Demo ist ein separates Beispielhaus. Sie zeigt die Bedienung, ohne auf meine Geräte, Konten oder Kameras zuzugreifen.

Seite	Inhalt
2	Architektur, Datenfluss und technische Entscheidungen
3	Bedienung, Tuya, Klima, Wetter und Strom
4	Google Home und Schalten ohne USB
5	Blink, Türregel und Alarmclips
6	Vom Linux-PC zum OVH-Server und zum iPhone
7	Sicherheit, Wartung, Tests und Fehlersuche
8	Kurzvortrag, Prüfungsfragen und Abgrenzung

Was ist praktisch bestätigt?

Das Dashboard lief auf OVH. Tuya, Live-Klima und sichtbare Blink-Kameras wurden dort bestätigt. Die Türregel wurde zuvor am echten System ausgelöst. Ein echter Blink-Clip wurde im Browser abgespielt und vor- und zurückgespult. Für die Google-Home-Bridge per WLAN sind Android-Build, Installation und Tailscale-Verbindung bestätigt. Der vollständige Stehlampentest vom iPhone bei ausgeschaltetem PC ist noch offen.

Planung und Entscheidungen lagen bei mir. Claude und Codex habe ich als Werkzeuge für Entwicklung, Dokumentation und Prüfung eingesetzt.

2 Architektur und Datenfluss

Eine Aktion im Browser wird zu einer HTTP-Anfrage an die API. Die API entscheidet anhand von Sitzung, Rolle und Gerätezuordnung, was passieren darf. Erst danach liest sie Daten oder ruft den passenden Anbieter auf. Bei Schaltaktionen wartet sie auf einen bestätigten Zustand.

Baustein	Aufgabe und Grund
React, TypeScript, Vite	Dashboard und Bedienung im Browser. Komponenten werden wiederverwendet, TypeScript prüft die Schnittstellen im Frontend.
ASP.NET Core 10	Die C#-Minimal-API verarbeitet Anfragen, prüft Berechtigungen und verbindet die externen Dienste. Sie liefert das gebaute Frontend aus.
EF Core und SQLite	Hausdaten, Zuordnungen und Verlauf bleiben in einer lokalen Datenbank. Für Klima gibt es eine eigene Datei. Migrationen halten Schemaänderungen nachvollziehbar.
REST und SSE	REST dient für Abfragen und Aktionen. Server-Sent Events übertragen Änderungen vom Backend zum Browser. Tuya wird weiterhin aktiv abgefragt.
Docker und Tailscale	Docker verpackt App und Laufzeit. Compose startet die Dienste. Tailscale stellt den privaten HTTPS-Zugang bereit.

Beispiel vom Schalter bis zum bestätigten Zustand

1. Die Oberfläche sendet den Schaltwunsch für ein konkret zugeordnetes Gerät.
2. Die API prüft Admin-Rolle, Online-Status und die unterstützte Schaltfunktion.
3. Der Tuya-Client oder die Android-Bridge führt den Befehl aus.
4. Der bestätigte Zustand wird zurückgemeldet. Fehlt die Bestätigung, zeigt die App den Fehler oder einen unbekannten Zustand.

Wo der Code liegt

frontend/src/ enthält Seiten, Komponenten und API-Aufrufe. **backend/HomeControl.Api/** enthält Endpunkte, Integrationen und Migrationen. **backend/HomeControl.Api.Tests/** enthält die xUnit-Tests. **integrations/** enthält die Bridge und weitere Anbindungen. **tools/** enthält Diagnose, Installation und die PDF-Generatoren.

3 Bedienung und Messwerte

Bereich	So nutze ich ihn
Übersicht	Zeigt Räume, Außentemperatur, Verbindungsstatus, Geräte, Batterien, Strom und Kameras. Online und Ein/Aus sind getrennte Zustände.
Räume und Geräte	Räume und Geräte verwalten, den richtigen Anbieter zuordnen und den Verlauf ansehen. Mit Tuya abgleichen holt den verfügbaren Stand.
Klima und Wetter	Die Klimaansicht zeigt Temperatur, Luftfeuchte und letzten Empfang. Open-Meteo liefert das Außenwetter. Ältere Raumwerte sind gekennzeichnet.
Reports und Waage	Reports zeigen gespeicherte Ereignisse und Tagesauswertungen. Die Waagenansicht braucht eine konfigurierte Quelle mit tatsächlich gemeldeten Messwerten.
Einstellungen	Je nach Rolle Konten, Verbindungen, Abgleich-Takte und Systeminformationen prüfen.

Tuya liest die Cloud und schaltet ausgewählte Geräte

Home Control signiert die Cloud-Anfragen, hält das Zugangs-Token im Speicher und fragt den Status regelmäßig ab. Die Takte lassen sich pro Gerät einstellen. Ein verknüpfter, erreichbarer Tuya-Schalter mit dem passenden booleschen Datenpunkt **switch_1** lässt sich als Admin bedienen. Danach liest das Backend den Gerätezustand zur Bestätigung erneut.

Watt, kWh und Batterie richtig lesen

Watt ist die aktuelle Leistung. Die aus Messreihen berechneten Kilowattstunden sind als Schätzung gekennzeichnet. Direkt gemeldete Shelly-Zählerstände erscheinen separat. Die PC-Steckdose bleibt als reiner Messpunkt gedacht und wird nicht geschaltet. Werte erscheinen nur, wenn die Quelle sie liefert. Ein gemeldeter Nullwert und ein fehlender Wert haben unterschiedliche Bedeutung.

Akkustände werden angezeigt, soweit sie der Anbieter liefert. Bei der geprüften Google-Home-Inventur lieferte keiner der vier Klimasensoren einen Akkustand. Das Projekt füllt solche Lücken nicht mit Schätzungen.

> Die Stromkarte ist umgesetzt. Ein eigener Haushaltszähler und aktuelle Messwerte für jedes Netzgerät sind nicht Teil des bestätigten Praxisstands.

4 Google Home und die Android-Bridge

Die Wohnzimmer-Stehlampe ließ sich in Google Home bedienen. Das verknüpfte Tuya-Projekt hatte auf ihren Lidl-Home-Sockel keinen Zugriff. Deshalb bleibt die Google-Home-Anbindung für diese Lampe getrennt vom Tuya-Schaltweg.

Wie Muju Home mit der API spricht

1. Muju Home nutzt das Google-Home-SDK auf dem Android-Handy und meldet Klima, unterstützte Schalter und Türsignale.
2. Für Schaltbefehle fragt die App das Backend ungefähr alle 1,5 Sekunden ab. Ein Befehl muss innerhalb von vier Sekunden abgeholt werden.
3. Die Android-App schaltet das verknüpfte Gerät und meldet den beobachteten Zustand. Abgelaufene Befehle werden nach einer Unterbrechung nicht nachgeholt.

Weg	Wann er gebraucht wird
Ursprünglich per USB	ADB leitet einen lokalen Port vom Handy zum PC weiter. Ein Gateway sendet von dort zur API. Dieser Weg diente zum Entwickeln und Testen.
Direkt per WLAN	Die App verwendet eine private HTTPS-Adresse auf OVH. Das Android-Handy braucht WLAN, Tailscale und die geöffnete Muju-Home-App. USB ist einmalig zur Installation nötig.
iPhone unterwegs	Das iPhone öffnet die Web-App mit Tailscale. Das Android-Handy zu Hause bleibt für die Google-Home-Anbindung zuständig.

Warum ein separater Gateway?

Der Gateway leitet nur vorgesehene Bridge-Anfragen an die API weiter. Die API erwartet einen gemeinsamen Bridge-Schlüssel und nimmt den Aufruf intern vom lokalen Gateway entgegen. Der HTTPS-Port für die Bridge wird nur im privaten Tailscale-Netz bereitgestellt.

> **Abnahmetest:** Auf dem Android-Handy den Gateway mit **/healthz** prüfen, in Muju Home das Zuhause wählen und USB abziehen. In Home Control auf den neuen Klima-Empfang warten. Danach den PC ausschalten und die Stehlampe vom iPhone per Mobilfunk ein- und ausschalten. Erst die bestätigte Reaktion belegt den vollständigen Weg.

5 Blink, Türregel und Alarmseite

Kamerabilder und Scharfschalten

Die Blink-Seite zeigt das letzte vorhandene Bild mit Aufnahmezeit. Als Admin kann ich ein neues Standbild anfordern. Die Livebild-Probe testet den Empfang von Videodaten. Eine Live-Wiedergabe im Browser ist dabei noch nicht umgesetzt.

1. In Szenarien den Türkontakt und das gewünschte Blink-System auswählen und die Regel aktivieren.
2. Ein frischer Wechsel von Zu auf Auf löst aus. Eine bereits offene Tür beim Start, doppelte Meldungen und alte Zustände nach einer Verbindungslücke lösen nicht aus.
3. Im Verlauf steht erst Bestätigt, wenn Blink das System beim erneuten Lesen als scharf meldet. Bei Bedarf wird es über die Blink-App wieder entschärft.

Warum Clips auf einer eigenen Seite stehen

Der Alarmverlauf zeigt das Ergebnis der Türregel. Die Clip-Liste kommt über Blink vom lokalen USB-Speicher des Sync-Moduls. Ein Clip gilt als zeitlich passend, wenn er zwischen 30 Sekunden vor und drei Minuten nach dem Ereignis aufgenommen wurde. Die Zeitnähe belegt keine Ursache.

Ein Video wird erst beim Abspielen über das Backend geladen. Ein echter Testclip brach in Chrome wegen eines leeren Videopakets ab. ffmpeg verpackt die heruntergeladene Kopie neu, ohne Bild und Ton neu zu kodieren. Der Stick wird dabei nur gelesen. Wiedergabe und Spulen wurden geprüft. Die Audiospur wurde technisch dekodiert, der hörbare Ton blieb ungeprüft.

Warum Blink auf OVH den Heimanschluss nutzt

Die gleiche OAuth-Testanfrage mit einem Platzhalter ergab vom Heimanschluss HTTP 401 und direkt von OVH HTTP 406. Der private Heimproxy leitete Blink danach erfolgreich über den Heimanschluss. Nur der Blink-HTTP-Client nutzt diesen Proxy. Sichtbare Kameras auf OVH wurden bestätigt.

> Blink verwendet eine inoffizielle Schnittstelle. Nur eine Instanz darf das rotierende Refresh-Token erneuern. Der Heimproxy braucht derzeit den eingeschalteten Linux-PC.

6 Betrieb auf OVH und auf dem iPhone

Warum wir umgezogen sind

Der VPS kann das Dashboard unabhängig vom Linux-PC bereitstellen. Die Haus- und Klima-Datenbanken wurden gesichert und in Docker-Volumes übernommen. Die Data-Protection-Schlüssel wurden mitgenommen, damit der Server das verschlüsselte Blink-Token lesen kann. Die Tuya-Zugangsdaten wurden separat in die Laufzeitkonfiguration übertragen.

Teil	Was er im Betrieb macht
Image	Node baut die React-Oberfläche. Das .NET SDK veröffentlicht die API. Im Laufzeit-Image liegen API, Frontend und ffmpeg.
Compose	Startet Home Control und optional den Sensor-Gateway. Beim Gateway-Start sind die Profile betrieb und sensoren aktiv.
Volumes	Unter /data bleiben Haus- und Klima-Daten erhalten. Unter /keys liegen die Schlüssel. Beide müssen gesichert werden.
Healthcheck	Der Endpunkt /healthz prüft den Dienst. Ein gesunder Container bestätigt noch keine erfolgreiche Kamera- oder Geräteanbindung.
Privater Zugang	Tailscale Serve stellt HTTPS für die Oberfläche und einen getrennten Eingang für die Android-Bridge bereit.

Der Startfehler beim Umzug

Das Quellarchiv war mit restriktiven Dateirechten entpackt worden. Der unprivilegierte App-Benutzer konnte appsettings.json im Image nicht lesen. Wir korrigierten die Leserechte für seine Gruppe und prüften den Containerstart aus einem privat entpackten Archiv in der CI.

Was vom iPhone erreichbar ist

Mit Tailscale verbunden, öffne ich die private Home-Control-Adresse in Safari. Über Teilen und Zum Home-Bildschirm lege ich einen Startpunkt ab. Die Anwendung läuft auf OVH. Direkt über Tuya verknüpfte Online-Geräte brauchen den Heim-PC dafür nicht.

> Die Abhängigkeiten gehören zum Betrieb: Google Home braucht das Android-Handy als Bridge. Blink braucht aktuell den Heimproxy auf dem Linux-PC. Für die Web-Oberfläche selbst genügt der erreichbare OVH-Server.

7 Sicherheit, Wartung und Fehlersuche

Wie der Zugriff geschützt wird

ASP.NET Core Identity verwaltet Konten und Passwort-Hashes. JWT und ein HttpOnly-Refresh-Cookie sichern die Sitzung. Admins dürfen verwalten und ausgewählte Aktionen ausführen. Lesende Nutzer und der API-Schlüssel für den Agenten erhalten keine Schaltrechte. Anfragen werden begrenzt. Private Adressen und Geheimnisse bleiben außerhalb des öffentlichen Repositorys.

Kamerabilder und Clips liefert die API an berechtigte Browser. Die Anbieter-Token bleiben beim Backend. Für Diagnose und Backups werden keine geheimen Werte in Terminalausgaben oder Git kopiert.

Was bei einer Störung zuerst geprüft wird

Symptom	Erster Schritt
Web-App nicht erreichbar	Tailscale-Verbindung prüfen. Auf dem Server Compose-Status und /healthz prüfen.
Klima oder Google-Schalter fehlen	WLAN und Tailscale auf dem Android-Handy, geöffnetes Muju Home, ausgewähltes Zuhause und letzten Empfang prüfen.
Tuya fehlt	Cloud-Konfiguration, Zugriff auf das Gerät und Status beim manuellen Abgleich prüfen. Online-Status ist nicht gleich Ein/Aus.
Blink fehlt	Heimproxy und die tokenführende Instanz prüfen. Bei HTTP 406 den Netzwerkweg mit einer Anfrage ohne echte Zugangsdaten vergleichen.
Altes Bild oder fehlender Clip	Aufnahmezeit oder Alarmverlauf prüfen. Standbilder, Clips und die Livebild-Probe sind verschiedene Funktionen.

Updates und Tests

Änderungen kommen über einen Branch und Pull Request ins Repository. Das Prüfskript kontrolliert Backend-Build, Migrationen, Tests, Frontend-Build und Lint. GitHub Actions prüft zusätzlich das Container-Image, den Sensor-Gateway und den Datenerhalt nach Neuerstellung.

Ein GitHub-Commit aktualisiert den VPS nicht automatisch. Vor einem Update Haus- und Klimadaten sowie Schlüssel sichern. Anschließend das Image wechseln, Healthchecks ausführen und die relevanten Geräte praktisch testen. Volumes nicht mit **docker compose down -v** löschen.

8 Kurzvortrag und Projektgrenzen

Fünf Minuten für die Abgabe

1. Ziel und Nutzen erklären. Eine Oberfläche zeigt Zustände verschiedener Hersteller und verbindet Geräteereignisse mit Regeln.
2. Die Demo öffnen. Räume, Klima und den Unterschied zwischen Online-Status und Schaltzustand zeigen.
3. Den Datenfluss erklären. React fragt die C#-API an. Diese prüft Rechte, liest SQLite und spricht mit dem jeweiligen Anbieter.
4. Das Türszenario erklären. Ein frisches Türsignal löst das Scharfschalten aus. Im Alarmverlauf steht das Ergebnis, daneben stehen zeitlich passende Clips.
5. Den OVH-Umzug und den Startfehler erklären. Image, Container und Volume unterscheiden. Zum Schluss sagen, welche Schritte an echten Geräten geprüft wurden.

```
docker compose --profile demo up demo
```

Die Demo läuft standardmäßig auf **localhost:5300**. Das Demo-Konto heißt **demo@example.de**. Das Standardpasswort lautet **Demo-Haus-2026**, sofern es nicht überschrieben wurde.

Typische Prüfungsfragen

Frage	Kurze Antwort
Warum ist ein grüner Test allein zu wenig?	Fakes prüfen Logik reproduzierbar. Geräte, Anbieter und Dateien können sich anders verhalten, wie beim echten Blink-Clip.
Warum SQLite und Volumes?	SQLite hält den Ein-Instanz-Betrieb einfach. Volumes trennen Daten und Schlüssel vom austauschbaren Image.
Was passiert bei einem Ausfall?	Die Oberfläche zeigt fehlende oder ältere Daten. Schaltzustände werden erst nach Bestätigung übernommen.
Was ist noch offen?	Der PC-aus-Test für die Google-Stehlampe, der n8n-Zugriff am echten Server, dauerhafter Blink-Betrieb ohne Heim-PC und weitere Regeln wie Wecker.

Weitere Details stehen bei Bedarf im GitHub-Repository. Dieses Handbuch deckt den Vortrag und die wesentlichen Betriebsabläufe ab.